

Department of Computer Science

University of Engineering and Technology Lahore Pakistan

هم نوا

Hamnawa

Accessible Document Reader for Visually Impaired Students

Project Proposal

Subject: Software Engineering **Group No.:** 2

Submitted by:

Muhammad Sarim	2025-CS-191
Abdullah Akbar	2025-CS-212
Muhammad Junaid Amir	2025-CS-193
Muhammad Ahmad Chaudhry	2025-CS-184

Submitted to:

Dr. Maimoona Salam

October 30, 2026

Document Control

Project	Hamnawa (ہم نوا) — Accessible Document Reader for Visually Impaired Students
Document	Project Proposal, version 1.0 (draft for supervisor review)
Course	Software Engineering, Department of Computer Science, UET Lahore
Group	Group 2
Supervisor	Dr. Maimoona Salam
Process model	Incremental model with iterative reviews
Status	For approval

In one sentence. Hamnawa turns the PDFs and slides that students receive every day into structured, spoken, voice-navigable content, so that a visually impaired student can study from the same material as everyone else, independently.

Contents

1 Introduction **3**

1.1 Background . . . 3

1.2 Problem Statement . . . 3

1.3 Why "Hamnawa"? . . . 3

1.4 Vision . . . 3

2 Objectives **3**

3 Scope **4**

3.1 What we will build (in scope) . . . 4

3.2 What we will *not* build (out of scope) . . . 4

3.3 Supported documents (definition) . . . 5

3.4 Assumptions and constraints . . . 5

4 Stakeholders and Users **5**

4.1 Illustrative personas . . . 5

5 Proposed Solution **6**

5.1 A day with Hamnawa . . . 6

5.2 Features by priority (MoSCoW) 6
5.3 What makes Hamnawa different 6

6 Preliminary Requirements **6**
6.1 Functional requirements 6
6.2 Non-functional requirements 7

7 Software Process Model **8**
7.1 Why incremental? 8
7.2 Planned increments 8

8 High-Level Architecture **8**

9 Requirements Elicitation and Validation Plan **9**

10 Feasibility Study **10**
10.1 Estimated resources 10

11 Risk Analysis **10**

12 Quality Assurance and Testing Plan **11**

13 Project Plan **12**
13.1 Schedule 12
13.2 Team organisation 12

14 Deliverables **13**

15 Expected Impact and Conclusion **13**

Glossary **13**

References **14**

1 Introduction

1.1 Background

Almost everything a university student learns from arrives as a document: lecture slides, handouts, past papers, research papers and textbooks. For most students, opening a PDF is effortless. For a student who is blind or has low vision, the same file can be a wall. Screen readers can read plain text, but many course files are not plain text. Slides have no clear reading order, multi-column layouts get read line by line across both columns, headers and page numbers interrupt every paragraph, and there is usually no quick way to jump to "the next section" or "question 4".

The result is that visually impaired students often depend on classmates, volunteers or manual re-typing to access course material. That costs them time, privacy and independence.

1.2 Problem Statement

Existing readers either read a document *linearly and literally*, or require the document to have been prepared accessibly in advance. Students cannot control how teachers prepare their files. There is no simple, free tool that takes an ordinary course document, recovers its *structure* (headings, lists, slides, questions), and lets the student *navigate it by voice and keyboard* the way a sighted reader skims a page.

1.3 Why "Hamnawa"?

Hamnawa (هم نوا) is an Urdu and Persian word meaning *one who shares your voice*: a companion, an ally. That is exactly the role we want the software to play. It does not replace the student's own effort; it stands beside the student and gives their study material a voice they can control.

1.4 Vision

Every document a classmate can open, a visually impaired student can open too — and move through just as freely.

2 Objectives

The project will be considered successful if it meets the following measurable objectives.

ID	Objective	How we will measure it
O1	Extract readable, correctly ordered text from text-based English PDF and PowerPoint (PPTX) files.	Reading-order accuracy on a test set of at least 20 course documents.

ID	Objective	How we will measure it
O2	Recover document structure (headings, paragraphs, lists, slides) and expose it as navigable sections.	Percentage of headings correctly detected on the test set.
O3	Read the content aloud with adjustable speed, voice, pause, resume and repeat.	All playback controls pass system tests.
O4	Allow complete use without a mouse: keyboard shortcuts and a small set of voice commands.	Every core task completed by a tester using keyboard only.
O5	Meet recognised accessibility guidance (WCAG 2.2 AA principles) and work with common screen readers.	Accessibility checklist and manual screen-reader testing.
O6	Follow a documented software engineering process and deliver the full set of engineering artifacts.	Artifacts reviewed against the checklist in Section 14.

3 Scope

3.1 What we will build (in scope)

- A web application that accepts text-based PDF and PPTX files.
- A structure-recovery engine that splits a document into headings, paragraphs, lists and slides.
- A reader with text-to-speech, adjustable speed and voice, pause, resume, repeat and skip.
- Navigation by section, heading level, slide or page, by keyboard and by simple voice commands.
- Bookmarks and a "resume where I stopped" feature.
- A fully accessible interface (labelled controls, logical focus order, high-contrast mode).

3.2 What we will *not* build (out of scope)

- Optical character recognition (OCR) for scanned documents.
- Automatic description of images, diagrams or mathematical equations.
- Urdu-language document reading or Urdu voice commands.
- Native mobile apps and account-based cloud storage.

These items are recorded as *future work* so that the scope stays honest and finishable.

3.3 Supported documents (definition)

For this project, a *supported document* is a text-based English PDF or PPTX file of up to **[e.g. 100 pages / 20 MB]**, containing selectable text. Anything else is detected and the user is told clearly why it cannot be read, instead of failing silently.

3.4 Assumptions and constraints

- Users have a modern desktop browser (the speech tools differ between browsers).
- All development tools are free and open source; the project budget is effectively zero.
- The project must be completed within the semester deadline: **[confirm date with supervisor]**.

4 Stakeholders and Users

Stakeholder	Interest in the system	Involvement
Visually impaired students	Primary users; need independent access to course material.	Interviews, usability testing
Low-vision students	Need large text, contrast and audio together.	Interviews, usability testing
Teachers and supervisors	Their ordinary files should just work; they approve the project.	Reviews, feedback
Disability / student-affairs office	Know real needs and existing support.	Requirements input
Development team	Deliver a correct, tested, well-documented product.	Build and test

4.1 Illustrative personas

These personas are *hypothetical* and will be replaced or refined after real interviews.

Ayesha second-year student, fully blind	Uses a screen reader on a laptop. Her teachers upload lecture slides as PDFs. She wants to jump straight to "Slide 12" and hear only the main points, and to resume the same file the next day.
Bilal third-year student, low vision	Can read large text slowly but tires quickly. He wants to hear long documents while following highlighted text, and to adjust speech speed to match his reading pace.

5 Proposed Solution

5.1 A day with Hamnawa

Ayesha opens Hamnawa and uploads this week's lecture slides. Within seconds the system announces: *"Operating Systems, Lecture 5. 24 slides, 6 sections."* She says *"next section"* and hears the section titles, picks section three, slows the voice down a little and listens. Halfway through she says *"bookmark"*, closes the laptop, and the next morning the system says: *"Welcome back. Resume from slide 14?"*

5.2 Features by priority (MoSCoW)

Priority	Features
Must have	Upload PDF/PPTX; text extraction; structure recovery; text-to-speech with speed control; pause/resume/repeat; keyboard navigation by heading and slide.
Should have	Voice commands (next, previous, repeat, slower, faster, bookmark); bookmarks and resume; high-contrast and large-text mode; section summary read-out of the first sentences.
Could have	Reading highlight for low-vision users; export of the structured text; reading history.
Won't have (this time)	OCR, image and equation description, Urdu support, mobile app.

5.3 What makes Hamnawa different

- **Structure-aware.** It does not just read text top to bottom; it understands that a document has sections and slides, and lets the user move between them.
- **Works with ordinary files.** Teachers do not need to prepare special accessible versions.
- **Hands-free by design.** Every task can be done without a mouse and without seeing the screen.
- **Free and lightweight.** Runs in a browser, with no paid services required.

6 Preliminary Requirements

These are the initial requirements; they will be refined and formalised in the Software Requirements Specification (SRS).

6.1 Functional requirements

ID	Requirement	Priority
FR1	The system shall let the user upload a supported PDF or PPTX document.	Must
FR2	The system shall extract text in the correct reading order and discard repeated headers, footers and page numbers.	Must
FR3	The system shall identify headings, paragraphs, lists and slide boundaries and build a navigable outline.	Must
FR4	The system shall read the content aloud and allow pause, resume, repeat and skip.	Must
FR5	The system shall allow the user to change speech speed and voice.	Must
FR6	The system shall support navigation to the next or previous section, heading or slide using the keyboard.	Must
FR7	The system shall accept a defined set of spoken commands for navigation and playback.	Should
FR8	The system shall let the user add bookmarks and resume from the last position.	Should
FR9	The system shall detect unsupported files and explain the reason by voice and text.	Must
FR10	The system shall offer a high-contrast, large-text display mode.	Should

6.2 Non-functional requirements

Quality	Requirement
Usability	A new user shall be able to open a document and start listening within 3 steps, using only the keyboard.
Accessibility	Every interface element shall have a text label, a logical focus order and a visible focus indicator, following WCAG 2.2 AA principles.
Performance	Reading shall start within **[e.g. 5]** seconds for a 50-page document; a voice command shall be acted on within 2 seconds.
Reliability	Failure to read a file shall never crash the application; the user always receives a clear spoken error message.
Privacy	Uploaded documents shall be processed in the user's own session and not shared; the limits of browser speech recognition shall be stated to the user.
Compatibility	The system shall work in current versions of Chrome and Edge, and with the NVDA screen reader.

Quality	Requirement
Maintainability	Code shall be modular (extraction, structuring, playback, interface) with documented interfaces.

7 Software Process Model

We will follow the **incremental process model** from Pressman and Maxim's *Software Engineering: A Practitioner's Approach*, applying the generic framework activities (communication, planning, modeling, construction, deployment) in each increment.

7.1 Why incremental?

- Requirements are partly unknown until we talk to real users, so a single big-bang plan (waterfall) would be risky.
- Each increment produces something that works and can be tested by a real user early.
- If time runs short, we still have a working core product, not a half-built whole.

7.2 Planned increments

Increment	Content	Outcome
1. Core reader	Upload a text-based PDF, extract text, read aloud with basic controls.	Working end-to-end skeleton
2. Structure	Heading, paragraph and slide detection; outline; navigation by keyboard; PPTX support.	Structure-aware navigation
3. Voice and memory	Voice commands, bookmarks, resume; display modes.	Hands-free, persistent experience
4. Polish and verify	Accessibility audit, user testing, bug fixing, documentation.	Release candidate

8 High-Level Architecture

The system is a pipeline of small, independent modules, which keeps each part testable and replaceable.

Technology direction (free and open source): a browser-based front end using the browser's built-in speech synthesis and recognition, and open-source libraries for PDF and PPTX parsing. Final choices will be recorded in the Software Design Document.

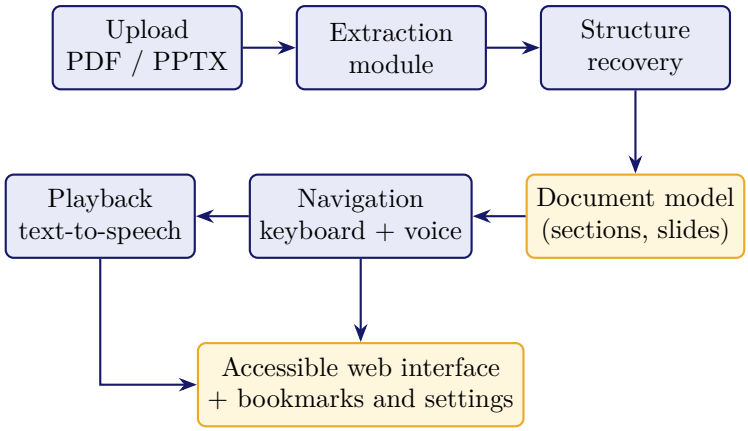


Figure 1: Pipeline architecture of Hamnawa: from uploaded file to a structured, spoken, navigable document.

9 Requirements Elicitation and Validation Plan

An accessibility tool must be shaped by the people who will use it. We will therefore:

1. **Interview** visually impaired or low-vision students and accessibility support staff (with informed consent) to understand current difficulties.
2. **Observe** how they currently study from PDFs and slides with their own screen readers.
3. **Build use cases and personas** from what we learn, replacing the hypothetical personas above.
4. **Review prototypes** at the end of every increment and record the feedback as change requests.
5. **Test usability** with real users or, if none are available, with sighted team members using the application with the screen turned off.

Ethics note. Participation will be voluntary, no personal data will be kept without consent, and feedback will be used only for improving the project.

10 Feasibility Study

Dimension	Assessment
Technical	Feasible. Text extraction from PDF/PPTX and browser speech synthesis use mature, free libraries. The hardest part (reading order and structure) is limited by our scope definition.
Economic	Feasible. Estimated cost is zero: free tools, free hosting tiers or local demo. Paid OCR and cloud voices are deliberately excluded.
Operational	Feasible. Runs in a browser, needs no installation, and fits into how students already use screen readers.
Schedule	Feasible if scope is respected. Incremental delivery ensures a working core even if later features slip.
Legal and ethical	Low risk. Only user-supplied files are processed; open-source licenses will be checked; user testing follows consent practice.

10.1 Estimated resources

Resource	Choice	Cost
Development	Free editors, frameworks and GitHub	Rs. 0
Speech output and input	Built-in browser speech tools	Rs. 0
Document parsing	Open-source PDF and PPTX libraries	Rs. 0
Hosting	Free tier or local demo	Rs. 0
Accessibility testing	Free screen readers (e.g., NVDA)	Rs. 0
Total		**Rs. 0**

11 Risk Analysis

Following the risk management approach in Pressman and Maxim (risk identification, projection, mitigation, monitoring and management).

ID	Risk	Prob.	Impact	Mitigation / contingency
R1	PDF layouts vary widely, so reading order is wrong.	High	High	Restrict scope to text-based documents; build a test corpus; show a clear warning for risky files.
R2	Browser speech recognition is inaccurate or unavailable.	Medium	Medium	Keep full keyboard control as the primary path; voice is a “should have”.

ID	Risk	Prob.	Impact	Mitigation / contingency
R3	No visually impaired user available for testing.	Medium	High	Contact accessibility staff early; fall back to screen-off testing by team members using NVDA.
R4	Scope creep (OCR, Urdu, images).	High	High	Fixed scope statement; change requests reviewed by the team lead.
R5	A team member is unavailable for a period.	Medium	Medium	Shared code ownership, documented modules, pair work on critical parts.
R6	Free hosting limits or outages during the demo.	Low	High	Prepare a local demo build and recorded backup demo.
R7	Open-source library has bugs or license issues.	Low	Medium	Evaluate libraries early; keep extraction behind a replaceable interface.

12 Quality Assurance and Testing Plan

- **Reviews and inspections:** formal peer review of the SRS, design and key code before each increment ends.
- **Unit testing:** extraction, structure recovery and command parsing tested with automated tests.
- **Integration testing:** the full pipeline from upload to speech on a test corpus of 20 or more course documents (slides, notes, papers).
- **System and acceptance testing:** every requirement is traced to at least one test case (a traceability matrix).
- **Accessibility testing:** keyboard-only walkthroughs, NVDA screen-reader checks and a WCAG-based checklist.
- **Metrics:** reading-order accuracy, heading-detection rate, response time of commands, defects found per increment.
- **Configuration management:** Git repository with branches per increment, tagged releases and issue tracking for defects and change requests.

13 Project Plan

13.1 Schedule

The schedule below is a draft in weeks and will be fitted to the real semester calendar. **[replace with actual deadline]**

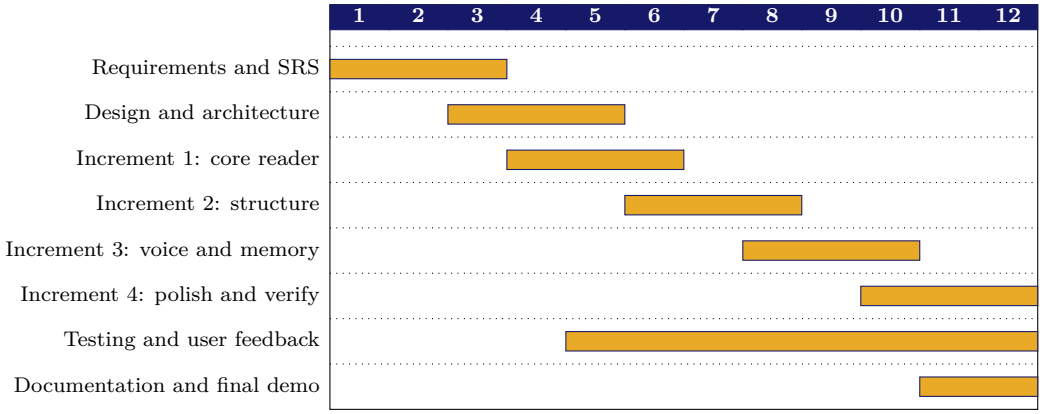


Figure 2: Draft project timeline (weeks).

13.2 Team organisation

Role	Responsibility	Assigned to
Project manager / coordinator	Plan, schedule, risk tracking, communication with the supervisor.	**[name]**
Requirements and documentation lead	Interviews, SRS, use cases, user manual.	**[name]**
Design and architecture lead	Architecture, UML models, design document.	**[name]**
Implementation lead	Core code, code reviews, repository health.	**[name]**
Quality and test lead	Test plan, test cases, accessibility testing, metrics.	**[name]**

With four members, some roles will be shared; every member takes part in both development and testing.

14 Deliverables

Artifact	Description	Phase
Project Proposal	This document.	Communication
Software Requirements Specification	Complete functional and non-functional requirements, use cases, personas.	Modeling
Analysis and design models	Use-case, class, sequence and activity diagrams; architecture description.	Modeling
Software Design Document	Module interfaces, technology choices, interface design.	Modeling
Project plan, risk register	Schedule, estimates, risk monitoring.	Planning
Source code and repository	Working increments with version history.	Construction
Test plan and test report	Test cases, traceability matrix, results, defect log.	Construction
User manual	Instructions for keyboard and voice use.	Deployment
Final report and demo	Working system, lessons learned.	Deployment

15 Expected Impact and Conclusion

Hamnawa addresses a real, everyday barrier in student life with a focused and achievable scope. Its success will be measured not by how many features it has, but by whether a visually impaired student can open an ordinary course document and study from it independently.

Beyond the product itself, the project gives our team an opportunity to practise the full software engineering lifecycle on a problem with real users, real constraints, and real consequences for usability and quality: eliciting requirements from people unlike ourselves, designing for accessibility, managing risk and verifying the result against measurable goals.

We request approval to proceed with **Hamnawa** (هم نوا) as our Software Engineering project.

Glossary

Screen reader Software that reads on-screen content aloud for blind or low-vision users (e.g., NVDA).

Text-to-speech (TTS) Technology that converts written text into spoken audio.

WCAG Web Content Accessibility Guidelines published by the W3C.

OCR Optical character recognition; converts images of text into text.

SRS Software Requirements Specification.

MoSCoW	Prioritisation scheme: Must, Should, Could, Won't.
Increment	A usable, tested portion of the product delivered in one cycle.

References

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

[2] W3C, *Web Content Accessibility Guidelines (WCAG) 2.2*, https://www.w3.org/TR/WCAG22/.

[3] NV Access, *NVDA screen reader*, https://www.nvaccess.org/.